

重点回答

HW1 挑战内容 4 · 本页内作答

1. SGLang 的 RadixAttention / RadixCache 解决什么问题？ 大模型服务中大量请求共享相同前缀（system prompt、few-shot 示例、多轮对话历史），传统"一请求一份 KV cache"的做法会把这部分前缀反复做 Prefill 计算，既浪费算力又推高 TTFT。RadixAttention 把已算出的 KV cache 用 **基数树（radix tree）按 token 前缀组织**：新请求到来时沿树做最长前缀匹配，命中部分直接复用 K/V，只对未命中的后缀做 Prefill；配合节点级引用计数与 LRU 驱逐，在显存受限时优先淘汰"最没有共享价值"的分支。一句话：**把跨请求的重复 Prefill 变成一次计算、多处复用**（本作业 HW2 实测：共享前缀组实际 Prefill token 减少 89.7%，TTFT p50 降低 64.9%）。

2. page-sized KV cache 与 prefix reuse 分别对应 pipeline 的哪一部分？ 二者都在"Prefill/Decode 之后、缓存管理之中"，但层次不同：**page-sized KV cache 属于显存管理层**（SGLang 的 `token_to_kv_pool_allocator`）：把 KV cache 池切成固定大小的页，按页分配/回收，配合请求级的页表（`req_to_token`）寻址，解决碎片化并让"部分复用"在物理上成为可能；**prefix reuse 属于调度与缓存匹配层**（`RadixCache.match_prefix`，在 `get_new_batch_prefill` 组 batch 时执行）：在树的逻辑视角上找到可复用的页序列，把它们登记为新请求的前缀，使本轮 Prefill 只算未命中部分。

3. vLLM 的 PagedAttention 解决什么问题？与 RadixAttention / paged KV 的联系与差异？ PagedAttention 解决的是 **KV cache 显存利用率**问题：早期系统按请求预留连续显存，造成大量内部/外部碎片，能并发的请求变少。它借鉴操作系统虚拟内存，把 KV cache 切成固定大小的 block，用 block table 间接映射，按需分配，几乎消除碎片，并天然支持前缀 block 共享与写时复制。**联系**：SGLang 的 page-sized allocator 与之同源（页式管理、页表寻址、页粒度共享）；**差异**：PagedAttention 的核心贡献在显存分页本身，前缀共享是附带能力（按 block 哈希链匹配）；RadixAttention 的核心贡献是把缓存组织成 radix tree，让**前缀复用成为默认的、自动的行为**——查找与序列长度同阶（而非与缓存规模），节点合并天然去重，且树结构与 LRU/引用计数结合可以在分支粒度上做驱逐。两者是互补关系：现代 SGLang 同时具备"页式物理管理 + 树式逻辑复用"。